

Linux + C++ Development Workflow

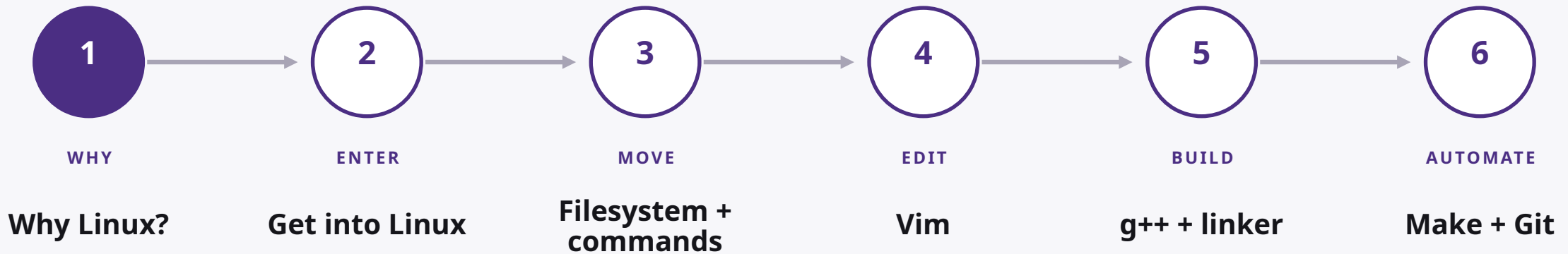
From source code to a running program

UW Bothell

C++

Linux

Today's path



You should leave able to SSH into Linux, navigate, edit, compile, link, and run a small C++ project.

Start with the end goal

A tiny C++ program is enough to expose the whole toolchain.

hello.cpp

```
#include <iostream>

int main() {
    std::cout << "Hello, world!\n";
    return 0;
}
```

Question:

How does this text file become a program the CPU can execute?

We'll answer that question on macOS first, then on Linux.

CLion is not the compiler

IDE

Editor, project view, debugger, buttons, integrations. CLion orchestrates tools for you.

Compiler

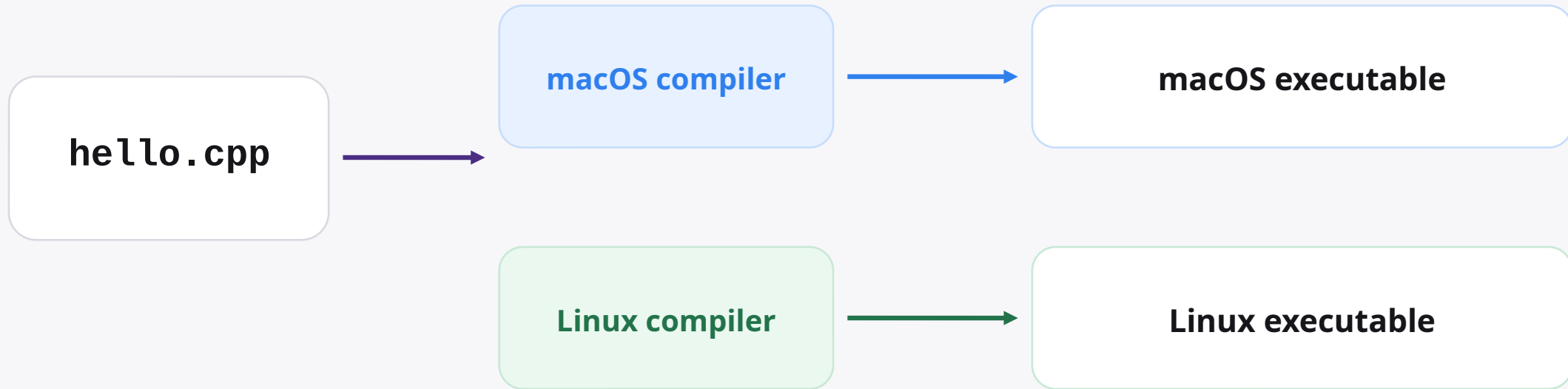
Translates C++ source into machine code or object code. On macOS this is often Apple Clang.

Operating system

Loads the executable, provides system services, files, processes, networking, and more.

On a Linux server, the GUI IDE may not be installed. The compiler still is.

Source is portable. Binaries usually are not.



Same source. Different target environment.

Java takes a different route: compile to bytecode, then run through a JVM for that platform.

Ways to get a Linux environment

UW lab

School-managed Linux machines. This is the course-aligned destination when available.

WSL 2

Linux environment integrated into Windows. Great for local practice.

Docker

Fast, disposable Linux containers. Useful, but not the same mental model as a full VM.

VM / Cloud

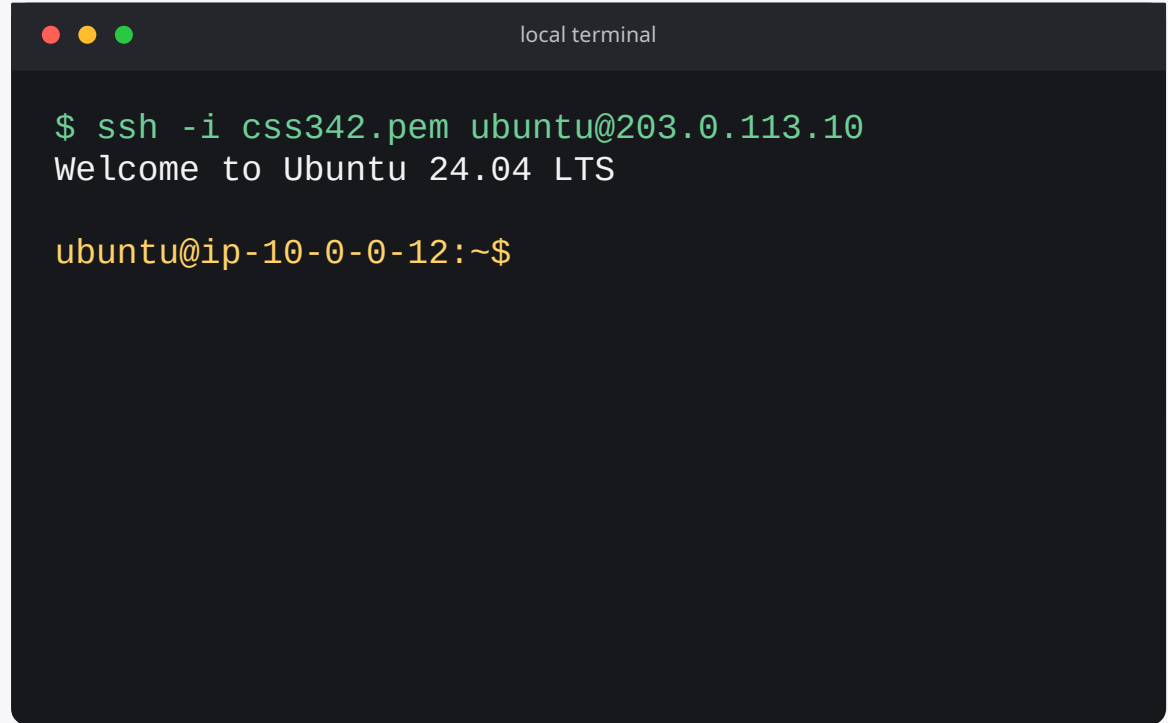
Ubuntu in VirtualBox, UTM, AWS EC2, or another cloud provider. Full machine model.

Today's demo: Ubuntu on AWS EC2 → SSH in → work from the shell.

A server can be boring

For this class, we only need four things:

- 1 **Ubuntu**
- 2 **Public IP / DNS**
- 3 **SSH allowed**
- 4 **A key pair**

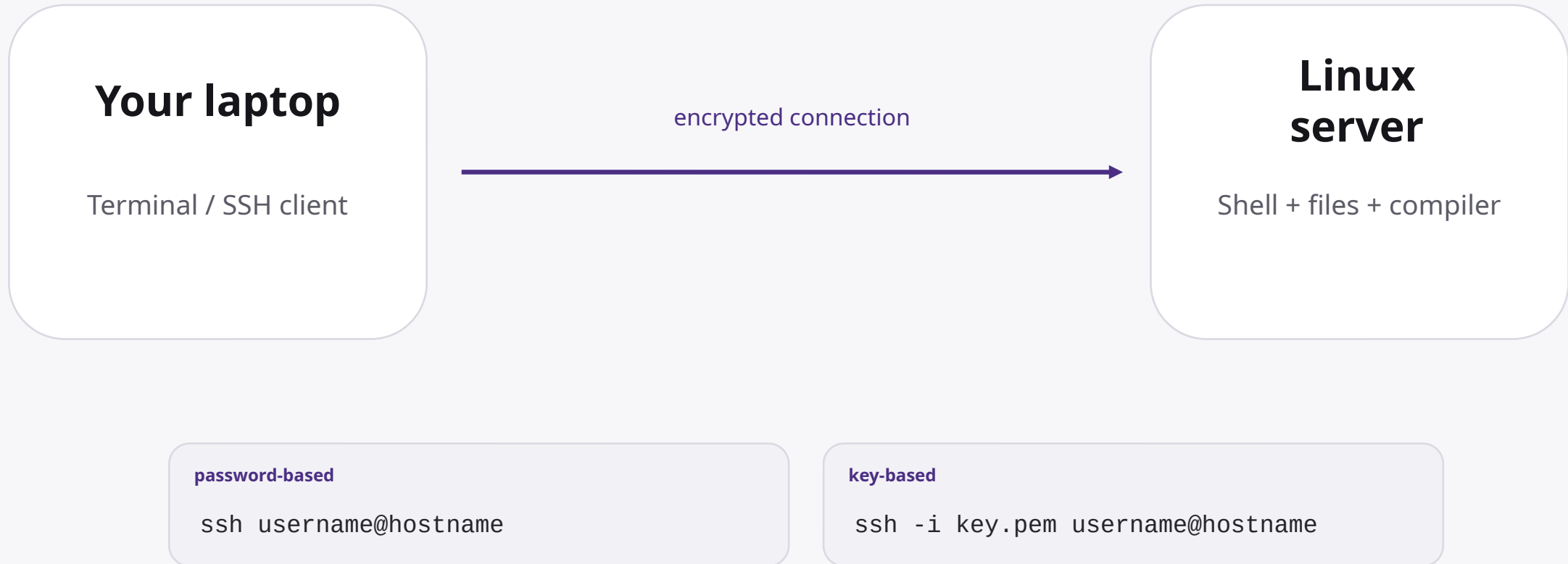
A terminal window titled "local terminal" with a dark background. It shows a green prompt character followed by the command `ssh -i css342.pem ubuntu@203.0.113.10`. The output is "Welcome to Ubuntu 24.04 LTS" in white. Below that, the prompt changes to `ubuntu@ip-10-0-0-12:~$` in yellow.

```
$ ssh -i css342.pem ubuntu@203.0.113.10
Welcome to Ubuntu 24.04 LTS

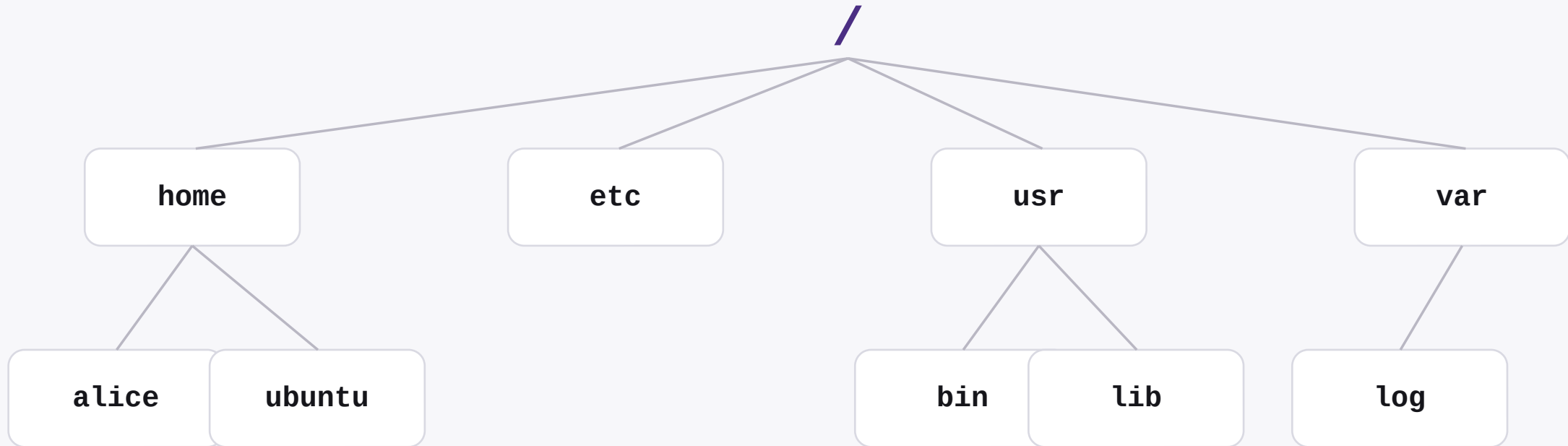
ubuntu@ip-10-0-0-12:~$
```

The cloud part ends here. From now on, it is just Linux.

SSH: your keyboard is here, the shell is there



Linux files form a tree



absolute path example

`/home/ubuntu/project/src/main.cpp`

Your home directory is usually `/home/<username>`. `~` is shorthand for your home.

First three commands

pwd

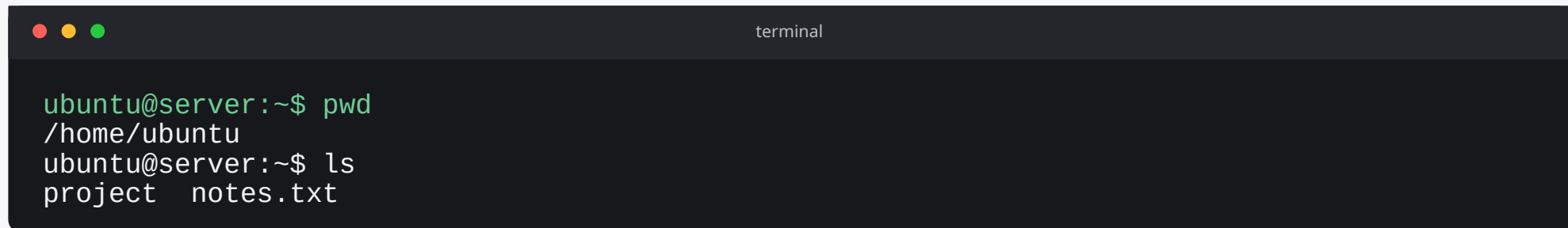
Where am I?

ls

What is here?

cd

Go somewhere else

A terminal window with a dark background and a title bar that says "terminal". The window has three colored window control buttons (red, yellow, green) in the top left corner. The terminal shows the following text:

```
ubuntu@server:~$ pwd
/home/ubuntu
ubuntu@server:~$ ls
project  notes.txt
```

Four path symbols worth memorizing

/

filesystem root

~

your home

.

current directory

..

parent directory

relative vs home

```
cd ../..  
cd ~/project
```

why ./ appears

```
./app  
./scripts/test.sh
```

A path is just directions through the tree.

Create, copy, rename, move

```
mkdir demo
```

make a directory

```
touch notes.txt
```

make an empty file

```
cp notes.txt copy.txt
```

copy

```
mv copy.txt renamed.txt
```

rename

```
mv renamed.txt demo/
```

move

mv does two jobs because “rename” is really moving a file to a new path.

Mini exercise: make a mess, then clean it up

1. build it

```
mkdir level1
cd level1
mkdir level2
cd level2
mkdir level3
cd level3
touch treasure.txt
```

2. remove it

```
cd ~/
rmdir level1      # fails

# remove empty dirs from inside out
# OR recursively remove the tree
```

Question: why can't rmdir remove level1 immediately?

Then introduce: `rm -r level1`

rm is not a recycle bin

```
terminal

$ rm notes.txt
$ rm -r old_project
$ rm -rf old_project

No "undo" command.
```

The famous T-shirt joke

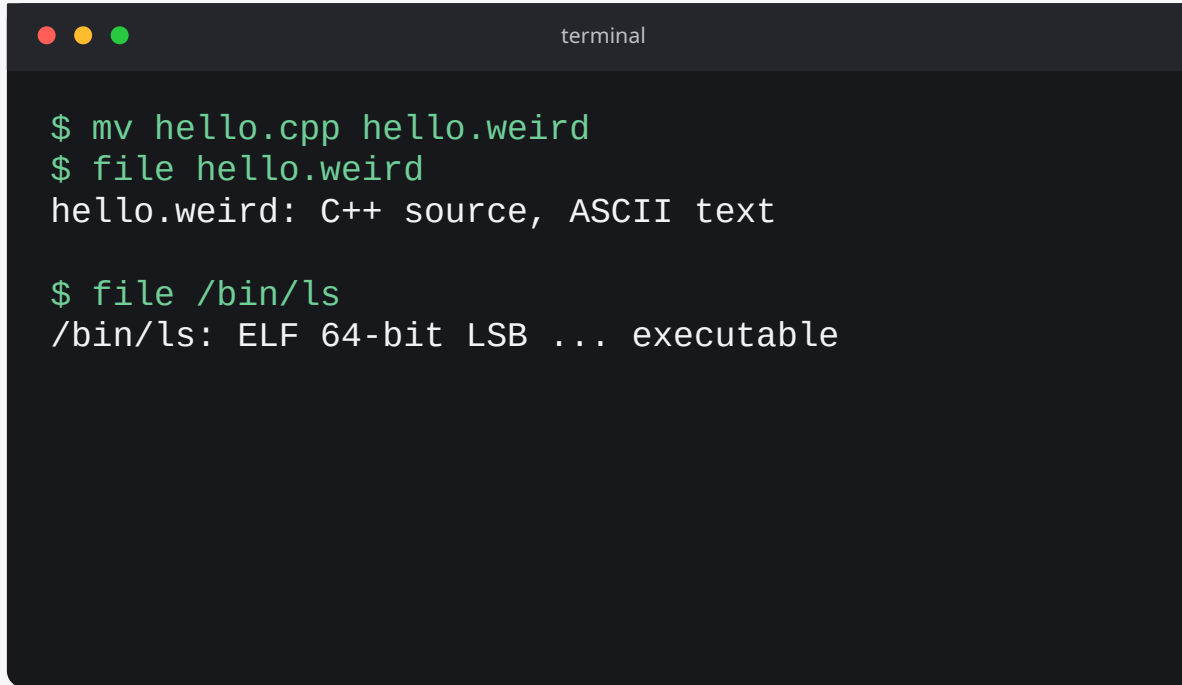
rm -rf /

Funny because recursive + forced deletion aimed at the root is catastrophic.

Safe habit: pwd → ls → then rm.

Practice destructive commands only in a throwaway directory.

Extensions are hints, not truth

A terminal window with a dark background and light green text. The title bar says "terminal". The commands and output are as follows:

```
$ mv hello.cpp hello.weird
$ file hello.weird
hello.weird: C++ source, ASCII text

$ file /bin/ls
/bin/ls: ELF 64-bit LSB ... executable
```

Linux usually does not decide what a file “is” from its extension.

- file inspects content/signatures
- Permissions determine whether a file can be executed
- Extensions are still useful conventions

Two ways to get code onto the server

SCP

Great for a one-off file copy. Uses SSH underneath.

Local → server:

```
scp hello.cpp ubuntu@host:~/
```

Server → local:

```
scp ubuntu@host:~/out.txt .
```

Git

Better for class projects and versioned code.

Clone once:

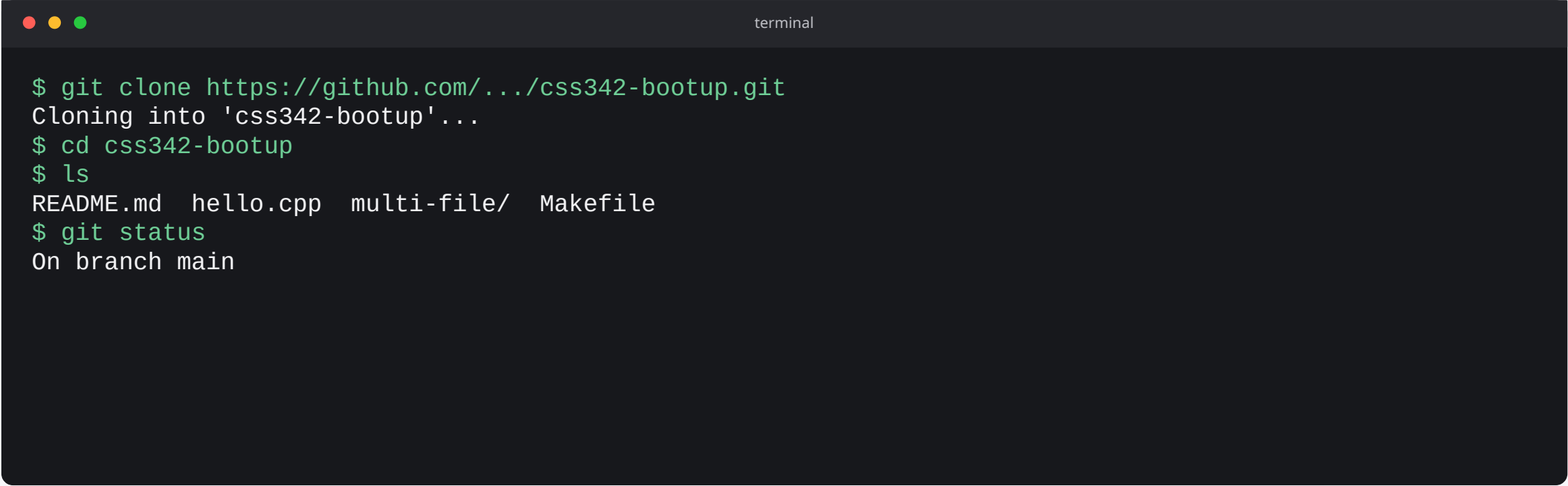
```
git clone <repo-url>
```

Then pull changes later:

```
git pull
```

We'll show SCP, but use Git for the class repository.

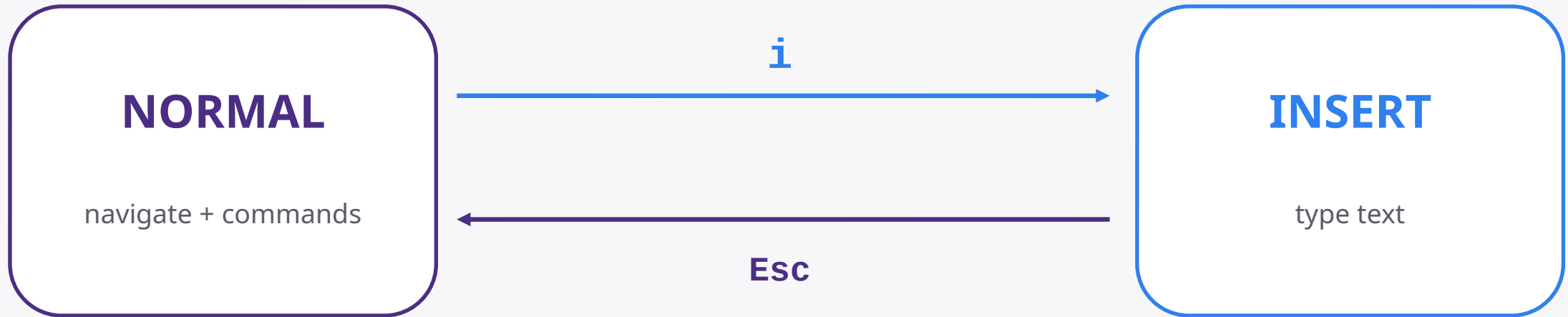
Git: get the class repository

A terminal window with a dark background and light green text. The window has three colored window control buttons (red, yellow, green) in the top-left corner and the word "terminal" in the top-right corner. The text inside the terminal shows the following commands and output:

```
$ git clone https://github.com/.../css342-bootup.git
Cloning into 'css342-bootup'...
$ cd css342-bootup
$ ls
README.md  hello.cpp  multi-file/  Makefile
$ git status
On branch main
```

For today: clone → inspect → edit → build. The repo is your take-home lab.

Vim has modes



The first skill is not “master Vim.” It is knowing which mode you are in.

Vim survival kit

i insert

Esc normal

:w save

:q quit

:wq save + quit

:q! quit, discard
changes

Goal: edit hello.cpp, change the output, save it, and get back to the shell.

A couple of “power user” moves

Ctrl-Z + fg

Ctrl-Z suspends the foreground process and returns you to the shell.

fg brings it back.

Useful job control, not a replacement for saving.

Split editing

```
:split file.cpp
```

```
:vsplit file.h
```

Ctrl-w w → move between windows

Enough to view a .h and .cpp side-by-side.

Try it:

job control

```
vim hello.cpp → Ctrl-Z → ls → fg
```

two files

```
:vsplit hello.h
```

```
Ctrl-w w
```

Now compile it yourself

```
terminal
$ g++ hello.cpp -o hello
$ ls -l
-rwxr-xr-x ... hello
-rw-r--r-- ... hello.cpp
$ ./hello
Hello, world!
```

g++ compiler driver

-o hello name the output

./hello run from this directory

This is the moment the entire session has been building toward.

One command is hiding several stages



show the object file

```
g++ -c hello.cpp  
# creates hello.o
```

then link

```
g++ hello.o -o hello  
# links executable
```

Multiple files make the linker visible

math.h

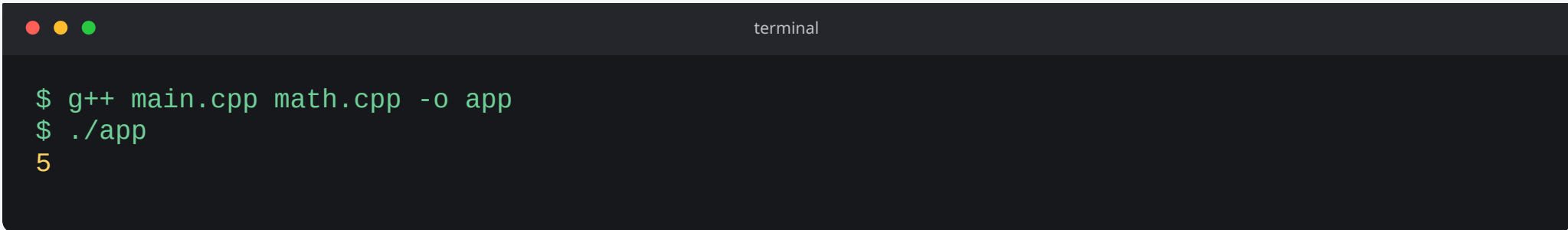
```
// math.h  
int add(int a, int b);
```

math.cpp

```
// math.cpp  
#include "math.h"  
int add(int a, int b) {  
    return a + b;  
}
```

main.cpp

```
// main.cpp  
#include "math.h"  
#include <iostream>  
int main() {  
    std::cout << add(2,3);  
}
```



```
terminal  
  
$ g++ main.cpp math.cpp -o app  
$ ./app  
5
```

Compiler error or linker error?

Compiler error

The compiler cannot turn a source file into object code.

Examples:

- syntax error
- unknown type/name
- wrong function call

Usually points to a source line.

Linker error

Object files exist, but they cannot be combined into the final executable.

Examples:

- undefined reference
- duplicate definition
- missing .cpp / library

Often not a syntax problem.

A useful debugging question: “Which stage failed?”

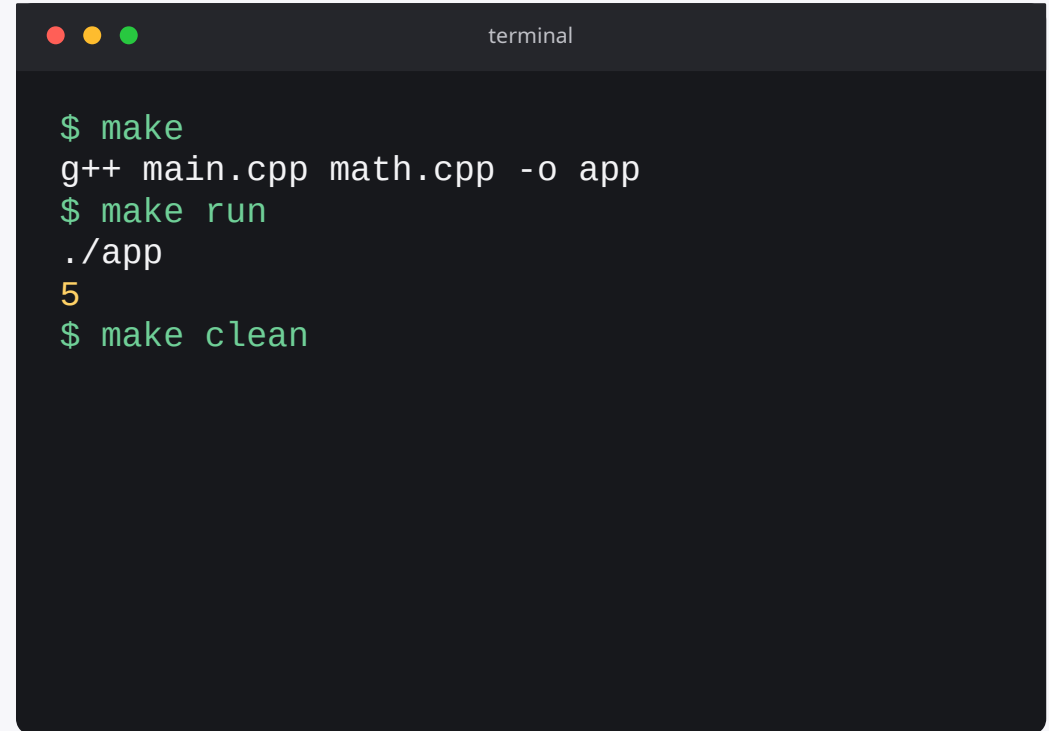
Make remembers the build command

Makefile

```
app: main.cpp math.cpp math.h
    g++ main.cpp math.cpp -o app

run: app
    ./app

clean:
    rm -f app
```

A terminal window with a dark background and light text. The title bar shows three colored circles (red, yellow, green) and the word "terminal". The terminal content shows a sequence of commands and their outputs: a green prompt "\$" followed by "make", then "g++ main.cpp math.cpp -o app", then a green prompt "\$" followed by "make run", then the output "./app", then a yellow "5", and finally a green prompt "\$" followed by "make clean".

```
terminal

$ make
g++ main.cpp math.cpp -o app
$ make run
./app
5
$ make clean
```

The first win: stop retyping the build command.

Later, Make can rebuild only what changed.

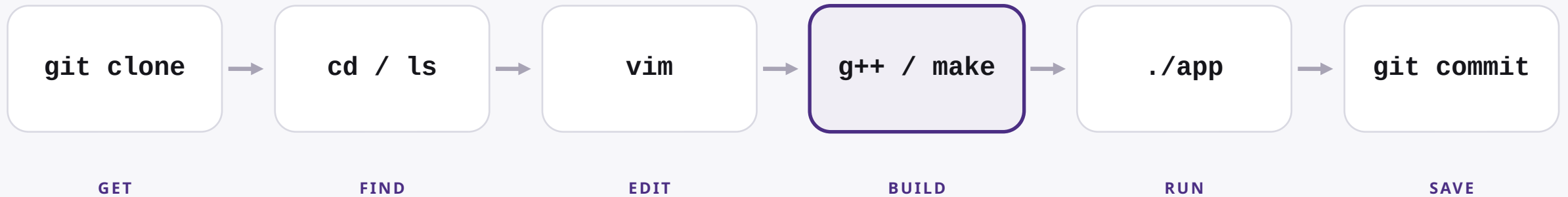
Save your work with Git

```
terminal
$ git status
modified: hello.cpp
$ git add hello.cpp
$ git commit -m "Update hello example"
[main 3e18c8a] Update hello example
$ git push
```

- 1 **status** what changed?
- 2 **add** stage it
- 3 **commit** name the checkpoint
- 4 **push** send it remote

A commit is a checkpoint, not a “final submission.” Commit often.

The complete workflow



When something breaks, ask where you are in the workflow.

Path problem? Editor problem? Compiler problem? Linker problem? Runtime problem?

Take-home challenge

Do this once without the slides:

- SSH into a Linux machine
- Clone the class repository
- Navigate to the multi-file example
- Edit one function in Vim
- Build with make and run it
- Use file on the executable
- Commit your change

suggested repo

```
css342-bootup/  
├── README.md  
├── hello/  
│   └── hello.cpp  
├── multi-file/  
│   ├── main.cpp  
│   ├── math.cpp  
│   ├── math.h  
│   └── Makefile  
└── exercises/  
    └── README.md
```

You do not need to memorize Linux. You need to be comfortable exploring it.

From source code to a running program.

That's the Linux workflow you need for CSS 342.

Questions?

one last time

```
git clone ...  
cd project  
vim main.cpp  
make  
./app  
git status
```